

喵呜乐消消

棋盘系统重构

BOARD SYSTEM REFACTOR · Claude Code 开发规格文档

V1.0 · 2026-05-22 · 微信小程序

覆盖问题：配额随机 · 坐标渲染 · 规整网格 · 牌堆系统 · 异形棋盘 · 安全区

一、文档概览与执行顺序

本文档覆盖棋盘系统的 6 个问题修复和新功能开发，Claude Code 按优先级顺序执行，每个问题独立成节，包含完整可运行代码。

优先级	问题	章节	预估工时	说明
P0	配额随机 + 可解性验证	第二章	1 天	当前难解关的根本原因，必须首先修复
P0	坐标渲染修复 + 安全区	第三章	0.5 天	防止牌溢出边界，和第二章同批修复
P1	半偏移规整网格	第四章	1 天	羊了个羊类型风格分布，视觉优化
P1	异形棋盘坐标系重构	第五章	1.5 天	用合法格子集合控制所有层的牌位
P2	牌堆系统	第六章	2 天	新增独立牌堆区，增加策略深度
P2	牌数种类调整	第七章	0.5 天	配合牌堆系统的数值平衡

执行约定 修改任何文件前先完整阅读对应章节。每个 P0 问题修复后立即在真机测试，确认通过后再进行下一步。不要合并多个问题一起改动。

二、配额随机 + 可解性验证 (P0)

当前问题：纯随机分配牌型导致某种图案数量不是 3 的倍数，出现理论上永远无法消除的死局。

核心思路 先按 3 的倍数分配配额，打乱后验证可解性，不通过则重新生成。通常 2-3 次内生成合法局面。

2.1 新建文件：utils/boardGenerator.js

将以下完整代码保存为新文件，供 game.js 引用。

```
// utils/boardGenerator.js
// 棋盘牌型生成器：配额随机 + 可解性验证

/**
 * 生成合法牌组
 * @param {number} totalCount - 牌总数 (48 / 54 / 60)
 * @param {number} typeCount - 图案种类数 (8 / 9 / 10)
 * @param {number} maxRetry - 最大重试次数，默认 10
 * @returns {number[]} 打乱后的牌型数组，每个元素是图案 ID (0~typeCount-1)
 */
function generateTiles(totalCount, typeCount, maxRetry = 10) {
  for (let attempt = 0; attempt < maxRetry; attempt++) {
    const tiles = buildQuotaTiles(totalCount, typeCount);
    shuffleArray(tiles);
    if (isSolvable(tiles, typeCount)) {
      console.log(`[BoardGen] 第${attempt+1}次生成成功`);
      return tiles;
    }
  }
  // 超过重试次数：返回最后一次生成结果（降级，不阻塞游戏）
  console.warn("[BoardGen] 超过重试次数，返回未验证局面");
  return buildQuotaTiles(totalCount, typeCount);
}

/** 步骤 1：按配额生成牌型（保证每种图案数量是 3 的倍数） */
function buildQuotaTiles(totalCount, typeCount) {
  const tiles = [];
  // 每种图案至少分配 3 张
  const basePerType = 3;
  let remaining = totalCount;

  // 先给每种图案分配基础 3 张
  for (let t = 0; t < typeCount; t++) {
    tiles.push(t, t, t);
    remaining -= 3;
  }
}
```

```

// 剩余数量按 3 的倍数随机补充
while (remaining >= 3) {
  const t = Math.floor(Math.random() * typeCount);
  tiles.push(t, t, t);
  remaining -= 3;
}

return tiles;
}

/** 步骤 2: Fisher-Yates 洗牌 */
function shuffleArray(arr) {
  for (let i = arr.length - 1; i > 0; i--) {
    const j = Math.floor(Math.random() * (i + 1));
    [arr[i], arr[j]] = [arr[j], arr[i]];
  }
}

/**
 * 步骤 3: 可解性验证 (贪心模拟)
 * 模拟游戏过程: 每次找 3 张可点击的同类牌消除
 * 能清空所有牌则视为可解
 */
function isSolvable(tiles, typeCount) {
  // 简化模拟: 假设收集槽无限大, 只验证配额正确性
  const counts = new Array(typeCount).fill(0);
  for (const t of tiles) counts[t]++;
  // 每种图案必须是 3 的倍数
  for (const c of counts) {
    if (c % 3 !== 0) return false;
  }
  return true;
}

module.exports = { generateTiles };

```

2.2 修改 game.js : 替换牌型生成逻辑

找到当前游戏中生成牌型的代码, 用以下方式替换:

```

// game.js 顶部引入
const { generateTiles } = require("../utils/boardGenerator");

// 在初始化关卡时调用 (替换原有的随机生成逻辑)
initLevel(levelConfig) {
  const { totalCount, typeCount } = levelConfig;

  // 原来的随机生成 (删除这段):
  // const tiles = Array(totalCount).fill(0).map(() =>

```

```
// Math.floor(Math.random() * typeCount));

// 替换为配额随机:
const tiles = generateTiles(totalCount, typeCount);

// 后续将 tiles 分配到棋盘格子（见第三章）
this.distributeTilesToBoard(tiles, levelConfig);
},
```

2.3 关卡配置表

以下为关卡段对应的 totalCount 和 typeCount，在关卡配置文件中维护：

关卡段	totalCount	typeCount	每种图案最少张数	备注
第 1 关（教学）	12	3	4 张	简单教学，图案少
第 2-10 关	48	8	6 张	基础关卡
第 11-20 关	54	9	6 张	新增猫爬架
第 21-30 关	60	10	6 张	新增猫罐头

三、坐标渲染修复 + 安全区 (P0)

当前问题：每层叠加都对牌的物理位置做偏移，层数多时累积偏移超出棋盘边界，导致边缘牌点不到。

根本原则 牌的 X/Y 坐标由行列号唯一确定，与层级无关。层级只影响 z-index 和 box-shadow，不改变任何物理位置。

3.1 坐标计算函数 (修改 game.js)

找到当前计算牌位置的代码，替换为以下函数：

```
// game.js

// 棋盘常量
const BOARD = {
  TILE_W: 60,    // 牌宽度 px (包含间距)
  TILE_H: 60,    // 牌高度 px
  GAP: 4,        // 牌间距 px
  SAFE_ZONE: 12, // 棋盘内边距 (安全区) px
  COLS: 8,       // 列数
  ROWS: 6,       // 行数
};

/**
 * 计算牌的屏幕坐标
 * @param {number} row    - 行号 (0~ROWS-1)
 * @param {number} col    - 列号 (0~COLS-1)
 * @param {number} layer  - 层级 (0=底层, 数字越大越靠上)
 * @returns {{ x, y, zIndex, boxShadow }}
 */
function getTileStyle(row, col, layer) {
  // X/Y 坐标与层级完全无关
  const x = BOARD.SAFE_ZONE + col * (BOARD.TILE_W + BOARD.GAP);
  const y = BOARD.SAFE_ZONE + row * (BOARD.TILE_H + BOARD.GAP);

  // 层级只影响 z-index 和阴影深度
  const zIndex = (layer + 1) * 10;
  const shadowY = Math.min(layer * 2, 6); // 最多 6px, 防止阴影过深
  const shadowBlur = Math.min(layer * 4, 12);
  const boxShadow = `0 ${shadowY}px ${shadowBlur}px rgba(0,0,0,${0.2 + layer*0.05})`;

  return { x, y, zIndex, boxShadow };
}
```

3.2 棋盘容器样式 (game.wxss)

找到棋盘容器的样式，添加以下关键属性：

```
/* game.wxss */
.board-container {
  position: relative;
  overflow: hidden;          /* 关键：裁切所有超出棋盘的内容 */
  /* 棋盘宽度 = 安全区×2 + 列数×(牌宽+间距) - 间距 */
  /* = 12×2 + 8×(60+4) - 4 = 536px */
  width: 536px;
  /* 棋盘高度 = 安全区×2 + 行数×(牌高+间距) - 间距 */
  /* = 12×2 + 6×(60+4) - 4 = 408px */
  height: 408px;
  background: /* 棋盘背景 */;
  border-radius: 16px;
}

.tile-item {
  position: absolute;
  width: 60px;
  height: 60px;
  border-radius: 10px;
  /* left/top/z-index/box-shadow 由 JS 动态设置 */
}
```

3.3 在 WXML 中绑定动态样式

```
<!-- game.wxml -->
<view class="board-container">
  <view
    wx:for="{{visibleTiles}}"
    wx:key="id"
    class="tile-item {{item.locked ? 'locked' : ''}}"
    style="left:{{item.x}}px; top:{{item.y}}px;
      z-index:{{item.zIndex}};
      box-shadow:{{item.boxShadow}};"
    bindtap="onTileTap"
    data-id="{{item.id}}"
  >
    <image src="{{item.imgSrc}}" mode="aspectFit"/>
  </view>
</view>
```

验证方法 修改完成后，故意生成一个全满棋盘（所有格子 4 层），检查最边缘格子（第 1 行/最后 1 行/第 1 列/第 8 列）的牌是否能正常点击，且无视觉溢出。

四、半偏移规整网格 (P1)

实现参考羊了个羊的奇偶行错位效果，视觉上更自然，同时保持坐标完全规整可计算。

4.1 修改 getTileStyle 函数

在第三章的 getTileStyle 函数基础上，添加偶数行偏移逻辑：

```
function getTileStyle(row, col, layer) {
  // 偶数行 (0,2,4) 不偏移，奇数行 (1,3,5) 向右偏移半格
  const rowOffset = (row % 2 === 1)
    ? (BOARD.TILE_W + BOARD.GAP) / 2
    : 0;

  const x = BOARD.SAFE_ZONE + col * (BOARD.TILE_W + BOARD.GAP) + rowOffset;
  const y = BOARD.SAFE_ZONE + row * (BOARD.TILE_H + BOARD.GAP);

  const zIndex    = (layer + 1) * 10;
  const shadowY    = Math.min(layer * 2, 6);
  const shadowBlur = Math.min(layer * 4, 12);
  const boxShadow = `0 ${shadowY}px ${shadowBlur}px rgba(0,0,0,${0.2+layer*0.05})`;

  return { x, y, zIndex, boxShadow };
}
```

4.2 奇数行列数调整

奇数行因为偏移半格，最右侧会有半格空缺。有两种处理方式：

方式	说明	推荐度
奇数行少放 1 列	奇数行只放 7 列 (0~6)，偶数行放 8 列 (0~7)	推荐，和羊了个羊完全一致
奇数行保持 8 列	最右侧牌会有一半超出棋盘，被 overflow:hidden 裁切	不推荐，裁切影响点击区域

采用推荐方式时，修改牌位分配逻辑：

```
function getColsForRow(row) {
  // 奇数行 7 列，偶数行 8 列
  return row % 2 === 1 ? 7 : 8;
}

// 计算总格子数 (用于验证牌总数是否匹配)
```


// 6 行: 3 行×8 格 + 3 行×7 格 = 24+21 = 45 格
// 注意: 45 格 < 48 张牌, 需要允许部分格子叠放 2 层

五、异形棋盘坐标系重构 (P1)

当前问题：上层的牌随机放置，无法约束在异形轮廓内，导致牌超出设计边界。

核心思路 用「合法格子集合」定义棋盘形状，所有层的牌只能放在合法格子内，从根本上保证异形轮廓正确。

5.1 新建文件：utils/boardShapes.js

将所有异形棋盘的合法格子集合定义在此文件中，后续新增异形棋盘只需在此添加。

```
// utils/boardShapes.js
// 异形棋盘合法格子集合定义
// 每个格子格式: { row: 行号, col: 列号 }
// 行列号从 0 开始

/** 标准棋盘: 6 行 8 列, 所有格子合法 */
function shapeStandard() {
  const cells = [];
  for (let r = 0; r < 6; r++)
    for (let c = 0; c < 8; c++)
      cells.push({ row: r, col: c });
  return cells;
}

/** 4 关, 14 关, 24 关
 * 爱心棋盘 */

function shapelove() {

  return [
    {row:1,col:2},{row:1,col:3},{row:1,col:6},{row:1,col:7},
    {row:2,col:1},{row:2,col:2},{row:2,col:3},{row:2,col:4},
    {row:2,col:5},{row:2,col:6},{row:2,col:7},{row:2,col:8},
    {row:3,col:1},{row:3,col:2},{row:3,col:3},{row:3,col:4},
    {row:3,col:5},{row:3,col:6},{row:3,col:7},{row:3,col:8},
    {row:4,col:2},{row:4,col:3},{row:4,col:4},
    {row:4,col:5},{row:4,col:6},{row:4,col:7},
    {row:5,col:3},{row:5,col:4},{row:5,col:5},{row:5,col:6},
    {row:6,col:4},{row:6,col:5},

  ];
}

/** (第 7, 17, 27 关: 折耳静坪)
 * 菱形棋盘 */
function shapediamond() {

  return [
    {row:1,col:4},{row:1,col:5},
    {row:2,col:3},{row:2,col:4},{row:2,col:5},{row:2,col:6},
    {row:3,col:2},{row:3,col:3},{row:3,col:4},
```

```
{row:3,col:5},{row:3,col:6},{row:3,col:7},
{row:4,col:2},{row:4,col:3},{row:4,col:4},
{row:4,col:5},{row:4,col:6},{row:4,col:7},
{row:5,col:3},{row:5,col:4},{row:5,col:5},{row:5,col:6},
{row:6,col:4},{row:6,col:5},
```

```
];
}
```

```
/** 双域棋盘（第10, 20, 30 关: 波斯双域）*/
```

```
function shapepersia() {
```

```
    return [
        ...[1,2,3,4,5,6].flatMap(r=>[1,2,3].map(c=>({row:r,col:c,zoneId:0}))),
        ...[1,2,3,4,5,6].flatMap(r=>[5,6,7,8].map(c=>({row:r,col:c,zoneId:1}))),
```

```
];
}
```

```
/** （第2, 12, 22 关: 白猫清浴）
```

```
 * 蝴蝶棋盘 */
```

```
function shapebutterfly() {
```

```
    return [
        {row:1,col:2},{row:1,col:3},{row:1,col:6},{row:1,col:7},
        {row:2,col:1},{row:2,col:2},{row:2,col:3},{row:2,col:4},
        {row:2,col:5},{row:2,col:6},{row:2,col:7},{row:2,col:8},
        {row:3,col:1},{row:3,col:2},{row:3,col:7},{row:3,col:8},
        {row:4,col:1},{row:4,col:2},{row:4,col:7},{row:4,col:8},
        {row:5,col:2},{row:5,col:3},{row:5,col:4},
        {row:5,col:5},{row:5,col:6},{row:5,col:7},
        {row:6,col:3},{row:6,col:4},{row:6,col:5},{row:6,col:6},
```

```
];
}
```

```
/** （第6, 16, 26 关）
```

```
 * 花瓣棋盘 */
```

```
function shapepetal() {
```

```
    return [
        {row:1,col:3},{row:1,col:4},{row:1,col:5},{row:1,col:6},
        {row:2,col:2},{row:2,col:3},{row:2,col:4},
        {row:2,col:5},{row:2,col:6},{row:2,col:7},
        {row:3,col:1},{row:3,col:2},{row:3,col:3},
        {row:3,col:6},{row:3,col:7},{row:3,col:8},
        {row:4,col:1},{row:4,col:2},{row:4,col:3},
        {row:4,col:6},{row:4,col:7},{row:4,col:8},
        {row:5,col:2},{row:5,col:3},{row:5,col:4},
        {row:5,col:5},{row:5,col:6},{row:5,col:7},
        {row:6,col:3},{row:6,col:4},{row:6,col:5},{row:6,col:6},
```

```
];
}
```

```
/** （第9, 19, 29 关）
```

```
 * 沙漏棋盘 */
```

```
function shapehourglass() {
```

```
    return [
        {row:1,col:1},{row:1,col:2},{row:1,col:3},{row:1,col:4},
        {row:1,col:5},{row:1,col:6},{row:1,col:7},{row:1,col:8},
        {row:2,col:2},{row:2,col:3},{row:2,col:4},
        {row:2,col:5},{row:2,col:6},{row:2,col:7},
        {row:3,col:3},{row:3,col:4},{row:3,col:5},{row:3,col:6},
        {row:4,col:3},{row:4,col:4},{row:4,col:5},{row:4,col:6},
        {row:5,col:2},{row:5,col:3},{row:5,col:4},
```

```

    {row:5,col:5},{row:5,col:6},{row:5,col:7},
    {row:6,col:1},{row:6,col:2},{row:6,col:3},{row:6,col:4},
    {row:6,col:5},{row:6,col:6},{row:6,col:7},{row:6,col:8},
  ];
}

/** 统一获取棋盘形状 */
const BOARD_SHAPES = {
  standard: shapeStandard,
  persia:    shapepersia,
  diamond:   shapediamond,
  love:      shapelove,
  hourglass: shapehourglass,
  butterfly: shapebutterfly,
  petal:     shapepetal,
};

function getBoardShape(shapeKey) {
  const fn = BOARD_SHAPES[shapeKey];
  if (!fn) { console.error("未知棋盘形状:", shapeKey); return shapeStandard(); }
  return fn();
}

module.exports = { getBoardShape };

```

5.2 修改牌位分配逻辑 (game.js)

所有层的牌从合法格子集合中选取，保证每层都在异形轮廓内：

```

const { getBoardShape } = require("../utils/boardShapes");

/**
 * 将牌组分配到棋盘各层
 * @param {number[]} tiles      - 牌型数组（由 generateTiles 生成）
 * @param {string}   shapeKey   - 棋盘形状键名
 * @param {number}   maxLayer   - 最大层数（建议 3~4）
 * @returns {Object[]} 每张牌的完整信息（row,col,layer,type,zoneId）
 */
function distributeTiles(tiles, shapeKey, maxLayer = 3) {
  const validCells = getBoardShape(shapeKey); // 合法格子集合
  const result = [];
  let tileIdx = 0;

  // 从底层到顶层，依次分配牌
  for (let layer = 0; layer <= maxLayer && tileIdx < tiles.length; layer++) {
    // 随机打乱合法格子，每层分配不同格子
    const shuffledCells = [...validCells].sort(() => Math.random() - 0.5);

    for (const cell of shuffledCells) {
      if (tileIdx >= tiles.length) break;
      result.push({

```

```
        id:      tileIdx,
        type:    tiles[tileIdx],
        row:     cell.row,
        col:     cell.col,
        layer:   layer,
        zoneId:  cell.zoneId ?? 0, // 异形分区 ID
        ...getStyle(cell.row, cell.col, layer), // x,y,zIndex,boxShadow
    });
    tileIdx++;
}
}

return result;
}
```

5.3 关卡与棋盘对照表

以下为 30 关的完整棋盘形状配置，在关卡配置文件中按此维护 shapeKey 字段。

关卡	shapeKey	有效点位	放牌上限	特殊规则
第 1 关	standard (标准矩形)	48 (6×8)	48	无，教学关
第 2 关	butterfly (蝴蝶)	30	30	无
第 3 关	standard	48	48	无
第 4 关	love (爱心)	30	30	无
第 5 关	standard	48	48	无
第 6 关	petal (花瓣)	30	30	无
第 7 关	diamond (菱形)	24	24	无
第 8 关	standard	48	48	无
第 9 关	hourglass (沙漏)	36	36	无
第 10 关	persia (波斯双域)	42	42	左右不跨域消除
第 11 关	standard	54	54	无
第 12 关	butterfly	30	30	无
第 13 关	standard	54	54	无
第 14 关	love	30	30	无
第 15 关	standard	54	54	无

关卡	shapeKey	有效点位	放牌上限	特殊规则
第 16 关	petal	30	30	无
第 17 关	diamond	24	24	无
第 18 关	standard	54	54	无
第 19 关	hourglass	36	36	无
第 20 关	persia	42	42	左右不跨域消除
第 21 关	standard	60	60	无
第 22 关	butterfly	30	30	无
第 23 关	standard	60	60	无
第 24 关	love	30	30	无
第 25 关	standard	60	60	无
第 26 关	petal	30	30	无
第 27 关	diamond	24	24	无
第 28 关	standard	60	60	无
第 29 关	hourglass	36	36	无
第 30 关	persia	42	42	左右不跨域消除

六、牌堆系统 (P2)

新增独立牌堆区，放置在主棋盘上方。每堆 5-8 张，只能从顶部逐张翻开，增加策略层次。

6.1 整体布局

界面从上到下新增牌堆区：



6.2 数据结构

```
// game.js — 在 initLevel 中初始化

// 牌堆配置 (在关卡配置文件中定义)
const PILE_CONFIG = {
  // 第 2-10 关: 3 个牌堆, 每堆 6 张
  "2-10": { count: 3, minCards: 5, maxCards: 6 },
  // 第 11-20 关: 3 个牌堆, 每堆 6-8 张
  "11-20": { count: 3, minCards: 6, maxCards: 8 },
  // 第 21-30 关: 4 个牌堆, 每堆 6-8 张
  "21-30": { count: 4, minCards: 6, maxCards: 8 },
};

// 牌堆状态 (在 data 中维护)
// piles: [
//   {
//     id: 0, // 堆 ID
//     cards: [3, 7, 2, ...], // 所有牌型 (从底到顶)
//     topIndex: 0, // 当前可翻开的牌的索引 (0=第一张)
//     isEmpty: false, // 是否已清空
//   },
//   ...
// ]
```

6.3 初始化牌堆

```
// game.js
initPiles(levelConfig, tileTypes) {
  const pileConf = getPileConfig(levelConfig.levelNum);
  const piles = [];

  for (let i = 0; i < pileConf.count; i++) {
    // 每堆随机数量
    const cardCount = pileConf.minCards +
      Math.floor(Math.random() * (pileConf.maxCards - pileConf.minCards + 1));

    // 生成本堆牌型（也用配额随机，保证 3 的倍数）
    const cards = generateTiles(cardCount, tileTypes.length);

    piles.push({
      id: i,
      cards: cards,
      topIndex: 0, // 从第 0 张开始翻
      isEmpty: false,
    });
  }

  this.setData({ piles });
},
```

6.4 点击牌堆逻辑

```
// game.js
onPileTap(e) {
  const pileId = e.currentTarget.dataset.pileId;
  const pile = this.data.piles[pileId];
  const slot = this.data.slot;

  // 边界检查
  if (!pile || pile.isEmpty) return; // 堆已空
  if (slot.length >= 7) return; // 收集槽已满，提示警告
  if (pile.topIndex >= pile.cards.length) return; // 防御性检查

  // 取出顶部牌
  const cardType = pile.cards[pile.topIndex];

  // 更新牌堆状态
  const newPiles = [...this.data.piles];
  newPiles[pileId] = {
    ...pile,
    topIndex: pile.topIndex + 1,
    isEmpty: pile.topIndex + 1 >= pile.cards.length,
  };

  // 加入收集槽（和点击主棋盘牌一样的逻辑）
  const newSlot = [...slot, { type: cardType, fromPile: pileId }];
```



```
this.setData({ piles: newPiles, slot: newSlot });
this.checkMatch(); // 检查三消
},
```

6.5 WXML 牌堆渲染

```
<!-- game.wxml: 在主棋盘上方添加牌堆区 -->
<view class="pile-zone">
  <view
    wx:for="{{piles}}"
    wx:key="id"
    class="pile-stack {{item.isEmpty ? 'empty' : ''}}"
    bindtap="onPileTap"
    data-pile-id="{{item.id}}"
  >
    <!-- 堆叠视觉: 最多显示3层背面 -->
    <view class="pile-back" wx:if="{{item.cards.length - item.topIndex > 2}}"/>
    <view class="pile-back" wx:if="{{item.cards.length - item.topIndex > 1}}"/>

    <!-- 顶部: 显示当前可翻牌的图案 -->
    <view class="pile-top" wx:if="{{!item.isEmpty}}">
      <image src="{{tileImages[item.cards[item.topIndex]]}" mode="aspectFit"/>
    </view>

    <!-- 已空状态 -->
    <view class="pile-empty-hint" wx:else>已清空</view>

    <!-- 剩余张数角标 -->
    <text class="pile-count">
      {{item.cards.length - item.topIndex}}
    </text>
  </view>
</view>
```

6.6 WXSS 牌堆样式

```
/* game.wxss */
.pile-zone {
  display: flex;
  flex-direction: row;
  justify-content: center;
  gap: 16px;
  padding: 12px 16px;
  background: rgba(255,140,66,0.06);
  border-bottom: 1px solid rgba(255,140,66,0.2);
}
.pile-stack {
  position: relative;
  width: 60px; height: 70px;
  cursor: pointer;
}
```

```
.pile-back {
  position: absolute;
  width: 56px; height: 56px;
  border-radius: 8px;
  background: linear-gradient(135deg, #2A1400, #3D2000);
  border: 1.5px solid rgba(255,140,66,0.25);
}
.pile-back:nth-child(1) { top: 8px; left: 2px; }
.pile-back:nth-child(2) { top: 4px; left: 1px; }
.pile-top {
  position: absolute;
  top: 0; left: 0;
  width: 60px; height: 60px;
  border-radius: 10px;
  background: #fff;
  border: 1.5px solid rgba(255,140,66,0.5);
  box-shadow: 0 4px 12px rgba(0,0,0,0.3);
  display: flex; align-items: center; justify-content: center;
}
.pile-count {
  position: absolute;
  bottom: 0; right: 0;
  background: #FF8C42;
  color: #fff;
  font-size: 11px; font-weight: 700;
  padding: 1px 6px;
  border-radius: 8px;
}
.pile-stack.empty { opacity: 0.35; pointer-events: none; }
```

七、牌数与图案种类调整 (P2)

加入牌堆系统后，总牌数增加，需同步调整主棋盘和图案配额。

关卡段	主棋盘牌数	牌堆数量	牌堆总牌数	总计	图案种类	每种图案最少
第 2-10 关	48 张	3 堆	约 18 张 (3×6)	约 66 张	8 种	每种≥6 张
第 11-20 关	54 张	3 堆	约 21 张 (3×7)	约 75 张	9 种	每种≥6 张
第 21-30 关	60 张	4 堆	约 26 张 (4×6.5)	约 86 张	10 种	每种≥6 张

注意 主棋盘和牌堆的牌型用同一个图案池，两者合并后每种图案数量仍需为 3 的倍数。在 initLevel 中先生成总量牌型，再按比例分配给棋盘和各牌堆。

7.1 总量生成 + 分配

```
// 修改后的 initLevel 逻辑
initLevel(levelConfig) {
  const { mainCount, typeCount, pileConf } = levelConfig;

  // 计算牌堆总需求
  const pileTotal = pileConf.count * Math.round(
    (pileConf.minCards + pileConf.maxCards) / 2 / 3
  ) * 3; // 取平均值并保证 3 的倍数

  // 生成足够所有区域使用的总牌型
  const totalTiles = generateTiles(mainCount + pileTotal, typeCount);

  // 前 mainCount 张分配给主棋盘
  const boardTiles = totalTiles.slice(0, mainCount);
  // 剩余分配给牌堆
  const pileTiles = totalTiles.slice(mainCount);

  // 分配棋盘
  const boardData = distributeTiles(boardTiles, levelConfig.shapeKey);
  // 分配牌堆
  this.initPiles(pileConf, pileTiles);
  this.setData({ boardTiles: boardData });
},
```

八、测试检查清单

完成每个模块后，使用以下清单验证，所有项目通过后方可提交。

8.1 P0 必须通过

编号	检查项	验证方法	预期结果
01	配额随机无死局	连续生成 100 局，检查是否每种图案数量都是 3 的倍数	全部通过，不出现余数为 1 或 2 的情况
02	边缘牌可点击	生成全满棋盘（4 层），测试第 1 列和第 8 列所有牌	所有边缘牌触摸响应正常
03	无溢出显示	查看棋盘 DOM，确认 overflow:hidden 生效	没有任何牌元素超出棋盘容器
04	安全区有效	最边缘牌距棋盘边缘≥12px	视觉上有明显间距，不贴边

8.2 P1 必须通过

编号	检查项	验证方法	预期结果
05	半偏移正确	查看棋盘，奇数行（1,3,5 行）是否向右错位半格	视觉上呈现错位网格效果
06	坐标系一致性	打印 isCoordAllowed(1,1,"love") 和原始代码对比	两者结果完全一致
07	爱心点位数	getShapeCells("love").length	等于 30
08	菱形点位数	getShapeCells("diamond").length	等于 24
09	沙漏点位数	getShapeCells("hourglass").length	等于 36
10	波斯双域点位数	getShapeCells("persia").length	等于 42
11	蝴蝶点位数	getShapeCells("butterfly").length	等于 30
12	花瓣点位数	getShapeCells("petal").length	等于 30
13	波斯双域左域	persia 中 col≤3 的格子 zoneId 均为 0	断言通过
14	波斯双域右域	persia 中 col>=5 的格子 zoneId 均为 1	断言通过
15	board_preview 渲染	在预览页选择「爱心」形状	正确显示爱心轮廓，空白格透明
16	波斯双域左右色	在预览页选择「波斯双域」	左域蓝色，右域绿色，col4 透明

编号	检查项	验证方法	预期结果
17	异形棋盘第 4，14，24 关	进入异形棋盘第 4，14，24 关（love），检查所有牌的位置	所有牌在爱心轮廓内，无超出

8.3 P2 必须通过

编号	检查项	验证方法	预期结果
18	牌堆初始化	进入有牌堆的关卡，检查堆数和张数	符合关卡配置，角标显示正确张数
19	逐张翻开	连续点击同一牌堆	每次只翻开 1 张，topIndex 递增，角标减少
20	清空状态	点击至最后一张后	堆变灰不可点击，显示「已清空」
21	总量平衡	检查主棋盘+所有牌堆的每种图案总数	每种图案总数仍为 3 的倍数

喵喵乐消消·棋盘系统重构开发规格文档 V1.0

请严格按章节顺序执行，每完成一个 P0 章节立即进行真机测试